Universidad de Caldas · Análisis y Diseño de Algoritmos · 2026-2

Proyecto ADA 2026-2

Red · Primera entrega

Selección de participantes y reparto del presupuesto de ruido

Formalización, NP-dureza por reducción, algoritmos con sus funciones de eficiencia y solución base ejecutable

Autor	**Jacobo Uribe Montoya**
Código	**36921**
Docente	Luz Enith Guerrero Mendieta
Versión del código	1.1.0 (archivo `VERSION`)
Fecha	Noviembre de 2026

Contenido

1 Introducción y alcance **3**

2 Formalización **3**

2.1 Parte 1: reparto óptimo del presupuesto (Rep) 4

2.2 Parte 2: selección de participantes (Sel) 4

2.3 Problema completo: las dos partes en secuencia 5

2.4 Versiones de decisión 5

3 NP-dureza de la selección **6**

3.1 Construcción de la reducción 6

4 Parte 1: reparto voraz con montículo **8**

4.1 Algoritmo 8

4.2 Optimalidad 8

4.3 Función de eficiencia 9

5 Parte 2: algoritmo exacto y voraz de aproximación **10**

5.1 Algoritmo exacto: árbol *incluir / no incluir* 10

5.2 Algoritmo voraz 11

5.3 Resumen de funciones de eficiencia 13

6 Solución base **13**

6.1 Organización del repositorio 13

6.2 Flujo de una ejecución 13

6.3 Verificación de restricciones 14

6.4 Correspondencia con la guía de la primera entrega 15

7 Resultados **15**

7.1 Caso de la profesora 15

7.2 Casos sintéticos de ejemplo 17

8 Validación experimental **17**

Referencias **19**

A Código de los algoritmos **19**

A.1 Parte 1: `src/red/reparto.py` 19

A.2 Parte 2: `src/red/seleccion.py` 21

1 Introducción y alcance

Una red de N nodos entrena un modelo de forma colaborativa, sin compartir datos crudos. Cada actualización se protege con ruido calibrado: más ruido protege más la privacidad, pero reduce la utilidad de la actualización. En cada ronda hay que tomar dos decisiones:

1. **Selección (Parte 2):** qué k nodos participan.
2. **Reparto (Parte 1):** cuántas unidades del presupuesto de ruido B recibe cada nodo elegido.

Las dos decisiones tienen dificultades muy distintas, y esa diferencia es la idea central del informe:

	Parte 1 · Reparto	Parte 2 · Selección
Decide	Cuántas unidades x_i recibe cada nodo	Cuáles k nodos participan
Dificultad	Voraz **exacto** (pseudo-polinomial)	**NP-difícil**: voraz aproximado
Algoritmo	Voraz con montículo, $O(n + B \log n)$ con n nodos a repartir ($n = k$ en el programa)	Voraz, $\Theta(Nk)$ evaluaciones; garantía $1 - 1/e$
Exacto de referencia	El propio voraz	Árbol incluir/no incluir, $\binom{N}{k}$ evaluaciones

Qué cubre esta primera entrega. La guía exige, para el Grupo 2: (i) la formalización de las dos partes del problema; (ii) la prueba formal de NP-dureza por reducción polinomial desde Cobertura Máxima; (iii) un repositorio con la solución base, sus dependencias, README e instrucciones; (iv) un manual técnico; y (v) ejecutar la solución base con el caso de la profesora, entregando resultados, tiempo y traza por caso. Todo se encuentra en el repositorio `Proyecto-ADA-2026-2`, cuya versión está en el archivo `VERSION` y se estampa en cada salida. La Sección 6.4 indica dónde está cada requisito de la guía.

Resultado principal: en el caso de la profesora ($N = 15$, $k = 5$, $B = 30$), ejecutado sobre el archivo tal como se recibió, el voraz elige N04, N15, N14, N13 y N06, cubre las 20 categorías (igual al óptimo exacto) y reparte el presupuesto en 7-5-6-5-7 con utilidad 99.5821, igual al óptimo por programación dinámica para esos cinco nodos.

2 Formalización

2.1 Parte 1: reparto óptimo del presupuesto (REP)

Definición 2.1 (Reparto óptimo del presupuesto)**. Entrada:** N nodos ya seleccionados; para cada nodo i, una utilidad $u_i : \mathbb{N} \to \mathbb{R}$ creciente y cóncava; un presupuesto $B \in \mathbb{N}$ en unidades enteras; opcionalmente, un tope c_i por nodo (su presupuesto de ruido tolerable).
Salida: un reparto $x = (x_1, \ldots, x_N) \in \mathbb{N}^N$.
Objetivo:

$$\max_x \sum_{i=1}^{N} u_i(x_i) \quad \text{sujeto a} \quad \sum_{i=1}^{N} x_i \le B, \qquad x_i \in \mathbb{N}, \qquad x_i \le c_i.$$

Ganancia marginal. La unidad número $x+1$ del nodo i aporta $g_i(x) = u_i(x+1) - u_i(x)$. *Creciente* significa $g_i(x) \ge 0$; *cóncava*, en el caso discreto, significa que las ganancias no aumentan: $g_i(0) \ge g_i(1) \ge g_i(2) \ge \cdots$.

Tamaño de la entrada. El par (N, B): N es el número de funciones u_i y de candidatos en cada decisión; B es el número de decisiones (una por unidad). Se supone que evaluar u_i cuesta tiempo constante. B cuenta por su *valor*, no por su número de bits; por eso la cota de la Sección 4 es pseudo-polinomial. En la solución completa, REP se aplica solo a los k nodos elegidos en la Parte 2, así que allí su tamaño es (k, B).

Forma de u_i (supuesto de modelado). El caso de la profesora da, por nodo, solo el coeficiente $w_i > 0$. Se adopta $u_i(x) = w_i \ln(1+x)$: vale 0 en $x = 0$, es creciente y es cóncava, con rendimientos decrecientes, que es lo que expresa el compromiso entre ruido y utilidad. El programa admite también $w_i\sqrt{x}$ y $w_i(1 - e^{-bx})$ (opción `-utilidad`) sin cambiar el algoritmo, porque la prueba de optimalidad solo usa la concavidad.

2.2 Parte 2: selección de participantes (SEL)

Definición 2.2 (Selección de participantes)**. Entrada:** nodos $V = \{1, \ldots, N\}$; una utilidad conjunta $f : 2^V \to \mathbb{R}_{\ge 0}$ evaluable en tiempo polinomial; un cupo $k \in \mathbb{N}$ con $1 \le k \le N$ (con $k = N$ participan todos).
Salida: un grupo $A \subseteq V$ con $|A| = k$.
Objetivo: $\max_{A \subseteq V,\ |A|=k} f(A)$.

Para $A \subseteq V$ y $j \notin A$, la ganancia marginal es $\Delta_j(A) = f(A \cup \{j\}) - f(A)$. Se exige que f sea **normalizada** ($f(\emptyset) = 0$), **monótona** ($\Delta_j(A) \ge 0$) y **submodular**: para todo $A \subseteq A' \subseteq V$ y $j \notin A'$, $\Delta_j(A) \ge \Delta_j(A')$. Es decir, un nodo aporta menos a un grupo grande que a uno pequeño. Al grupo grande se le llama A', y no B, para no confundirlo con el presupuesto.

Utilidad implementada: cobertura. El nodo j aporta un conjunto S_j de categorías de datos de un universo U, y

$$f(A) = \Big|\, C(A) \,\Big|, \qquad C(A) = \bigcup_{j \in A} S_j.$$

Una categoría aportada por dos nodos cuenta una sola vez, igual que los datos redundantes.

Tamaño de la entrada. (N, k) y dos costos de evaluación. En la implementación cada S_j es un entero con un bit por categoría ($w = 64$ bits por palabra):

- $\tau = O(|U|/w)$: una ganancia marginal $\Delta_j(A)$, cuando lo ya cubierto $C(A)$ está guardado (lo que hace el voraz);
- $\tau_f = O(k\,|U|/w)$: $f(A)$ calculada desde cero, uniendo los k conjuntos de A (lo que hace cada hoja del algoritmo exacto).

2.3 Problema completo: las dos partes en secuencia

Definición 2.3 (Ronda de entrenamiento)**. Entrada:** los nodos con sus conjuntos S_j y sus utilidades u_j, el cupo k y el presupuesto B. **Salida:** un grupo A con $|A| = k$ y un reparto x con $x_j = 0$ si $j \notin A$ y $\sum_j x_j \le B$. **Criterio:** primero se maximiza la cobertura $f(A)$ (Parte 2) y luego, con A fijo, la utilidad $\sum_{j \in A} u_j(x_j)$ (Parte 1).

La solución base resuelve la Parte 2 con el voraz de aproximación y la Parte 1 con el voraz exacto. Cuando dos nodos aportan las mismas categorías nuevas, el voraz de selección prefiere el de mayor $p_j = u_j(1) - u_j(0)$, la ganancia de su primera unidad de presupuesto (proporcional a w_j): así, a igual cobertura, entran los nodos que más aprovechan el reparto. Optimizar la cobertura y la utilidad a la vez queda fuera de esta entrega.

2.4 Versiones de decisión

Para hablar de P y NP se agrega un umbral t y la pregunta se vuelve de sí o no:

Problema	Pregunta
Rep	¿Existe un reparto con $\sum_i x_i \le B$ y $\sum_i u_i(x_i) \ge t$?
Sel	¿Existe $A \subseteq V$ con $\lvert A\rvert = k$ y $f(A) \ge t$?

Observación 2.4 (Hacia dónde va la reducción)**.** Rep se resuelve de forma exacta con el voraz de la Sección 4; la dificultad combinatoria está en elegir el grupo, así que la reducción va hacia Sel. Con $B = 0$ el problema completo es exactamente Sel (no hay nada que repartir), de modo que la NP-dureza de Sel se traslada al problema completo.

3 NP-dureza de la selección

Definición 3.1 (Reducción polinomial)**.** $X \leq_p Y$ si existe una función φ, calculable en tiempo polinomial, que transforma cada instancia I de X en una instancia $\varphi(I)$ de Y tal que I tiene respuesta *sí* $\iff$ $\varphi(I)$ tiene respuesta *sí*. Si X es NP-difícil y $X \leq_p Y$, entonces Y es NP-difícil.

Observación 3.2 (Dirección)**.** Para probar que *nuestro* problema es difícil, se reduce **desde** un problema conocido como difícil **hacia** el nuestro: si SEL se resolviera en tiempo polinomial, componiendo con φ se resolvería el problema conocido. La dirección contraria no demostraría nada.

Definición 3.3 (Cobertura Máxima, MAXCOVER)**.** Dados un universo U, conjuntos $S_1, \ldots, S_m \subseteq U$ y enteros k y t: ¿existen k conjuntos cuya unión tenga al menos t elementos?

Cadena hasta un problema clásico. Cobertura de Vértices es NP-completo (Karp, 1972) y se reduce a SETCOVER (universo = aristas; el conjunto del vértice v = sus aristas incidentes). SETCOVER se reduce a MAXCOVER tomando $t = |U|$. Por lo tanto MAXCOVER es NP-difícil. Se escoge MAXCOVER porque tiene la misma estructura que SEL: elegir k objetos cuyas contribuciones se solapan.

3.1 Construcción de la reducción

Dada $I = (U, S_1, \ldots, S_m, k, t)$ con $1 \leq k \leq m$, se construye $\varphi(I) = (V, f, k', t')$ como sigue.[1]

Componente	Se construye	Por qué
Nodos	$V = \{1, \ldots, m\}$, $N = m$	Escoger el nodo i equivale a escoger S_i
Utilidad	$f(A) = \left\lvert \bigcup_{i \in A} S_i \right\rvert$	Cuenta exactamente lo que cuenta MAXCOVER
Cupo y umbral	$k' = k$, $t' = t$	Mismo número de elecciones, misma meta

V son los nodos (lo que se *escoge*) y U los elementos (lo que se *cubre*). En el repositorio, la construcción es `desde_cobertura_maxima` (`src/red/seleccion.py`).

[1] Si $k > m$ se toman todos los conjuntos y se decide comparando $|\bigcup_i S_i|$ con t; si $k = 0$ la respuesta es *sí* solo cuando $t \leq 0$. En ambos casos φ devuelve una instancia fija con esa misma respuesta.

Lema 3.4 (La instancia construida es válida)**.** Para $A \subseteq V$ y $j \notin A$, $\Delta_j(A) = |S_j \setminus C(A)|$. En consecuencia, f es normalizada, monótona y submodular.

Demostración. $C(A \cup \{j\})$ es la unión disjunta de $C(A)$ y $S_j \setminus C(A)$, de donde sale la fórmula. Normalizada: $f(\emptyset) = 0$. Monótona: $|S_j \setminus C(A)| \geq 0$. Submodular: si $A \subseteq A'$ entonces $C(A) \subseteq C(A')$, luego $S_j \setminus C(A') \subseteq S_j \setminus C(A)$ y por lo tanto $\Delta_j(A') \leq \Delta_j(A)$. ■

Lema 3.5 (φ es polinomial)**.** Copiar los conjuntos cuesta $\Theta(m + \sum_i |S_i|)$ y evaluar f cuesta $\Theta(|U| + \sum_{i \in A} |S_i|)$. Ambos son lineales en el tamaño de I, y la instancia construida no crece de forma exponencial.

Teorema 3.6 (φ conserva la respuesta)**.** I tiene respuesta *sí* en MaxCover $\iff$ $\varphi(I)$ tiene respuesta *sí* en Sel.

Demostración. ($\Rightarrow$) Si $|S_{i_1} \cup \cdots \cup S_{i_k}| \geq t$, el grupo $A = \{i_1, \ldots, i_k\}$ cumple $|A| = k$ y $f(A) \geq t$. ($\Leftarrow$) Si existe A con $|A| = k$ y $f(A) \geq t$, los conjuntos $\{S_i : i \in A\}$ son k conjuntos con unión de al menos t elementos. Si la versión de MaxCover admite *a lo sumo* k conjuntos, se completan hasta k y, por monotonía, la unión no baja. ■

Las dos direcciones son necesarias. Con solo ($\Rightarrow$), una φ que siempre produjera instancias *sí* pasaría la prueba sin ser una reducción. La dirección ($\Leftarrow$) garantiza que φ no crea soluciones que no existían.

Teorema 3.7. Sel es NP-difícil y su versión de decisión es NP-completa.

Demostración. MaxCover es NP-difícil y MaxCover $\leq_p$ Sel por los tres resultados anteriores. Además Sel $\in$ NP: un grupo A es un certificado que se verifica en tiempo $O(k\,|U|)$. ■

Ejemplo. $U = \{1, \ldots, 6\}$, $S_1 = \{1, 2, 3\}$, $S_2 = \{3, 4\}$, $S_3 = \{4, 5, 6\}$, $S_4 = \{1, 4\}$, $k = 2$, $t = 6$. De los seis pares, solo $\{1, 3\}$ cubre los 6 elementos: ambos problemas responden *sí* con el mismo testigo. Si se cambia S_3 por $\{4, 5\}$, el elemento 6 no aparece en ningún conjunto y ambos responden *no*. Una prueba automática comprueba la equivalencia en 150 instancias aleatorias.

Observación 3.8. La NP-dureza no demuestra que todo algoritmo exacto sea exponencial: demuestra que un algoritmo exacto polinomial implicaría P=NP. Por eso la selección se resuelve con el voraz de la Sección 5, que tiene garantía de aproximación.

4 Parte 1: reparto voraz con montículo

4.1 Algoritmo

En esta sección N es el número de nodos entre los que se reparte; en la solución completa son los k elegidos, de modo que allí todas las cotas se leen con $N = k$. Se entregan las B unidades una por una, siempre al nodo cuya *siguiente* unidad aporta más. Un montículo de máximos guarda la próxima ganancia de cada nodo; la ganancia $g_i(x)$ se calcula solo cuando se necesita, nunca como una tabla de $N \times B$ valores.

Algoritmo 1: RepartirVoraz$(u_1, \ldots, u_N,\ B,\ c)$ — `repartir_voraz`

Entrada: utilidades cóncavas y crecientes u_i; presupuesto B; topes opcionales c_i
Salida: reparto $x_1, \ldots, x_N$ con $\sum_i x_i \le B$

$x_i \leftarrow 0$ para todo i;
$H \leftarrow$ montículo de máximos con $(g_i(0),\, i)$ para cada i con $c_i > 0$; ▷ $O(N)$
para $b \leftarrow 1$ **hasta** B **hacer**
  $(g, i) \leftarrow$ máximo de H;
  si $g \le 0$ **entonces terminar** ▷ `nadie aporta ya nada`;
  $x_i \leftarrow x_i + 1$;
  si $x_i = c_i$ **entonces**
    sacar i de H ; ▷ $O(\log N)$
  si no
    reemplazar el máximo por $(g_i(x_i),\, i)$; ▷ $O(\log N)$
devolver x;

En Python, `heapq` es un montículo de *mínimos*: se guarda $(-g,\, i)$ para que la mayor ganancia salga primero, y el índice i rompe los empates a favor del menor. Reemplazar el máximo se hace en una sola operación, `heapreplace`.

4.2 Optimalidad

Teorema 4.1 (Optimalidad del voraz)**.** Si cada u_i es cóncava y creciente, el Algoritmo 1 devuelve un reparto de utilidad máxima.

Demostración. (1) La utilidad es una suma de ganancias. Por suma telescópica, $\sum_i u_i(x_i) = \sum_i u_i(0) + \sum_i \sum_{y=0}^{x_i - 1} g_i(y)$. El primer término no depende del reparto, así que maximizar la utilidad equivale a maximizar la suma de las ganancias usadas.

(2) El voraz toma las mayores ganancias. Por concavidad, la próxima ganancia de cada nodo es la mayor que le queda, y el montículo extrae la mayor entre todos los nodos. Sea λ la menor ganancia que tomó el voraz: todo lo tomado es $\ge \lambda$ y todo lo no tomado es $\le \lambda$.

(3) Intercambio. Sea x^* un reparto óptimo distinto del voraz x^g, ambos con la misma suma (sin topes se puede suponer $\sum_i x_i^* = B$, porque las ganancias son ≥ 0; si el voraz se detiene antes porque ya nadie aporta, las unidades restantes no cambian la utilidad). Hay un nodo i con $x_i^* > x_i^g$ y otro j con $x_j^* < x_j^g$. Mover una unidad de i a j cambia la utilidad en

$$g_j(x_j^*) - g_i(x_i^* - 1) \;\geq\; \lambda - \lambda = 0,$$

porque la unidad x_j^* de j fue tomada por el voraz y la unidad $x_i^* - 1$ de i no. El nuevo reparto sigue siendo óptimo y su distancia a x^g baja en 2.

(4) Repitiendo el intercambio se llega a x^g sin perder utilidad. ■

Observación 4.2 (Por qué se exige concavidad)**.** Con ganancias que suben, el voraz falla. Dos nodos, $B = 2$: el primero gana 1 y luego 10; el segundo gana 2 y luego 0. El voraz da una unidad a cada uno (utilidad 3), pero lo óptimo es dar las dos al primero (utilidad 11). La prueba automática correspondiente reproduce este caso.

4.3 Función de eficiencia

Con constantes $c_1, \ldots, c_6 > 0$ del modelo RAM y un montículo binario de altura $h = \lfloor \log_2 N \rfloor$:

Línea	Operación	Costo	Veces
1	inicializar x_i	c_1	N
2	calcular $g_i(0)$ y construir el montículo	$\leq c_2 N$	1
4–5	leer el máximo y comprobar $g > 0$	c_3	$\leq B$
6–7	$x_i \leftarrow x_i + 1$ y revisar el tope	c_4	$\leq B$
8 o 10	sacar i, o calcular $g_i(x_i)$ y reubicarlo	entre c_5 y $c_5 h$	$\leq B$

Sumando el peor caso (cada reubicación recorre toda la altura):

$$T_{\text{peor}}(N, B) = (c_1 + c_2)N + (c_3 + c_4)B + c_5\, B \lfloor \log_2 N \rfloor \;\in\; \Theta(N + B \log N),$$

y si cada reubicación termina tras una comparación (por ejemplo, ganancias iguales), $T_{\text{mejor}}(N, B) \in \Theta(N + B)$. Por lo tanto

$T(N, B) \in O(N + B \log N)$ y $\Omega(N + B)$; si $B \geq N$, $\Theta(N + B \log N) = \Theta(B \log N)$, que es la cota del enunciado. En la solución completa se reparte entre los k elegidos: $O(k + B \log k)$.

Pseudo-polinomialidad. B se escribe con $\lceil \log_2(B + 1) \rceil$ bits, pero el voraz hace B pasos: es polinomial en el *valor* de B y exponencial en su *longitud*, igual que la programación dinámica $O(nW)$ de la mochila 0/1. Como el voraz toma exactamente las ganancias mayores que un umbral

λ, podría buscarse λ por bisección y obtener una cota polinomial en la longitud de B cuando las ganancias son enteras y acotadas; con $u_i(x) = w_i \ln(1+x)$ las ganancias son reales, así que este informe no usa ese argumento y la cota que se demuestra es la pseudo-polinomial.

Presupuesto continuo. Si $x_i \in \mathbb{R}_{\geq 0}$, en el óptimo todos los nodos que reciben presupuesto quedan con la misma ganancia marginal λ (nivelación o *water-filling*). Con $u_i(x) = w_i \ln(1+x)$ resulta $x_i = \max(0,\ w_i/\lambda - 1)$.

5 Parte 2: algoritmo exacto y voraz de aproximación

5.1 Algoritmo exacto: árbol *incluir / no incluir*

Como SEL es NP-difícil, el exacto se usa solo como referencia para medir al voraz en casos pequeños (`seleccion_exacta`).

Algoritmo 2: SELECCIÓNEXACTA$(S_1, \ldots, S_N,\ k)$ — `seleccion_exacta`

```
1 función Explorar(i, A):
2     si |A| = k entonces devolver (f(A), A);                ▷ caso 1: hoja
3     si N − i < k − |A| entonces devolver (−∞, ∅);          ▷ caso 2: poda
4     (v1, A1) ← Explorar(i + 1, A ∪ {i}) ;                  ▷ caso 3a: incluir i
5     (v2, A2) ← Explorar(i + 1, A) ;                        ▷ caso 3b: no incluir i
6     devolver la mejor de las dos;
7 devolver Explorar(0, ∅);
```

Deducción 1: sin cupo, por recurrencia y sustitución

Si se recorren todos los subconjuntos, cada llamada con n nodos por decidir hace dos llamadas con $n-1$ y trabajo constante c, y cada hoja evalúa f desde cero con costo τ_f:

$$\begin{aligned} T(n) &= 2T(n-1) + c, \qquad T(0) = \tau_f \\ &= 2\big(2T(n-2) + c\big) + c = 2^2 T(n-2) + c\,(2+1) \\ &= 2^3 T(n-3) + c\,(2^2 + 2 + 1) \\ &\ \ \vdots \\ &= 2^j T(n-j) + c \sum_{l=0}^{j-1} 2^l = 2^j T(n-j) + c\,(2^j - 1). \end{aligned}$$

Con $j = n$ se llega al caso base:

$T(N) = 2^N\tau_f + c\,(2^N - 1) = (\tau_f + c)\,2^N - c \;\in\; \Theta(2^N)$ **si** τ_f **es constante;** $\Theta(2^N\tau_f)$ **en general.**

Deducción 2: con cupo k, recurrencia de Pascal

Sea $L(n, r)$ el número de hojas (evaluaciones de f) cuando quedan n nodos por decidir y faltan r por escoger. La rama *incluir* gasta un lugar del cupo y la rama *no incluir* no:

$$L(n, r) = L(n-1, r-1) + L(n-1, r), \qquad L(n, 0) = 1, \qquad L(n, r) = 0 \text{ si } r > n.$$

Es la regla de Pascal, $\binom{n}{r} = \binom{n-1}{r-1} + \binom{n-1}{r}$, con los mismos casos base; por inducción sobre n, $L(n, r) = \binom{n}{r}$. El algoritmo evalúa f **exactamente** $\binom{N}{k}$ veces, y una prueba automática lo comprueba en 100 instancias. Como cada llamada interna lleva a una hoja y la profundidad es a lo sumo N:

$$T_{\text{exacto}}(N, k) \in \Omega\Big(\tbinom{N}{k}\tau_f\Big), \qquad T_{\text{exacto}}(N, k) \in O\Big(\tbinom{N}{k}(N + \tau_f)\Big),$$

y el mejor y el peor caso coinciden, porque el recorrido no depende de los datos.

Lema 5.1. $\binom{N}{\lfloor N/2 \rfloor} \geq \dfrac{2^N}{N+1}$.

Demostración. Los $N+1$ números $\binom{N}{0}, \ldots, \binom{N}{N}$ suman 2^N, y el mayor de ellos, $\binom{N}{\lfloor N/2 \rfloor}$, es al menos su promedio. ∎

Con $k \approx N/2$ el exacto hace $\Omega(2^N/N)$ evaluaciones: es exponencial. Con k fijo, $\binom{N}{k} \in \Theta(N^k)$, un polinomio cuyo grado crece con k; como k es parte de la entrada, el algoritmo no es polinomial en general. Por eso el programa solo lo ejecuta si $\binom{N}{k} \leq 500\,000$.

5.2 Algoritmo voraz

Algoritmo 3: SELECCIÓNVORAZ$(S_1, \ldots, S_N,\ k)$ — `seleccion_voraz`

```
A ← ∅; cubierto ← ∅;
para paso ← 1 hasta k hacer
    para cada j ∉ A hacer
        Δ_j ← |S_j \ cubierto| ;                ▷ una evaluación de la ganancia
    j* ← el j de mayor Δ_j (en empate, el de mayor p_j; luego, el de menor índice);
    A ← A ∪ {j*}; cubierto ← cubierto ∪ S_j*;
devolver A;
```

En el código, la ganancia es `(masks[j] & ~cubierto).bit_count()`: las categorías de j que aún no están cubiertas, contadas como bits.

Desempate. $p_j = u_j(1) - u_j(0)$ es la ganancia de la primera unidad de presupuesto del nodo j; con $u_j(x) = w_j \ln(1+x)$, $p_j = w_j \ln 2$, así que gana el de mayor coeficiente. El desempate solo elige entre candidatos con el mismo Δ_j máximo, de modo que la sucesión de valores $f_1, f_2, \ldots$ y la garantía de abajo no cambian; comparar p_j cuesta $O(1)$, así que el conteo de evaluaciones tampoco cambia. Sí cambia *qué* grupo se entrega cuando hay varios con la misma cobertura: en el caso de la profesora, 33 grupos distintos cubren las 20 categorías, y su utilidad tras el reparto va de 91.06 a 107.04. Desempatando por índice, el voraz entregaba uno de 92.58; desempatando por p_j, entrega uno de 99.58 (Sección 7).

Conteo de evaluaciones. En el paso i quedan $N - i + 1$ candidatos:

$$T_{\text{voraz}}(N,k) = \sum_{i=1}^{k} (N - i + 1) = kN - \frac{k(k-1)}{2}.$$

Como $k \le N$, $k - 1 < N$ y $\frac{k(k-1)}{2} < \frac{kN}{2}$, así que $\frac{kN}{2} \le T_{\text{voraz}} \le kN$:

$T_{\text{voraz}}(N,k) \in \Theta(Nk)$ evaluaciones y $\Theta(Nk\tau)$ de tiempo, con $\tau = O(|U|/w)$. El mejor y el peor caso coinciden.

Teorema 5.2 (Nemhauser, Wolsey y Fisher, 1978)**.** Si f es normalizada, monótona y submodular, el grupo A_g del voraz cumple $f(A_g) \ge \left(1 - \frac{1}{e}\right) f(A^*) \approx 0.632\, f(A^*)$.

Idea de la demostración. Sea $\text{OPT} = f(A^*)$ y f_i el valor tras i pasos. Por monotonía, $\text{OPT} \le f(A_i \cup A^*)$, con A_i el grupo del voraz tras i pasos. Por submodularidad, esa diferencia se reparte en a lo sumo k aportes de nodos de A^*, así que alguno aporta al menos $(\text{OPT} - f_i)/k$, y el voraz toma uno al menos igual de bueno: $\text{OPT} - f_{i+1} \le (1 - \frac{1}{k})(\text{OPT} - f_i)$. Tras k pasos, $\text{OPT} - f_k \le (1 - \frac{1}{k})^k \text{OPT} \le \text{OPT}/e$, porque $1 - y \le e^{-y}$. ■

Aplicación. Las hipótesis son las del Lema 3.4. En cada caso pequeño el programa calcula $f(A_g)/f(A^*)$ y verifica que sea $\ge 1 - 1/e$; las pruebas lo comprueban en 300 instancias aleatorias. La cota no se puede mejorar en tiempo polinomial salvo que P=NP (Feige, 1998), así que el voraz alcanza la mejor garantía posible.

El voraz no siempre es óptimo. Con $S_1 = \{1,2,3,4\}$, $S_2 = \{1,2,5\}$, $S_3 = \{3,4,6\}$ y $k = 2$ (caso `casos/ejemplos/ej00_tres_nodos.json`), el voraz toma S_1 y luego S_2 o S_3 (los dos aportan un elemento nuevo), con $f = 5$, pero $\{S_2, S_3\}$ cubre 6. La razón $5/6 \approx 0.833$ cumple la garantía.

5.3 Resumen de funciones de eficiencia

Algoritmo	Función de eficiencia	¿Mejor = peor?
Reparto voraz (P1)	peor $\Theta(N + B \log N)$; mejor $\Theta(N + B)$	No
Selección exacta con cupo (P2)	$\binom{N}{k}$ evaluaciones; $O(\binom{N}{k}(N + \tau_f))$	Sí
Selección exacta sin cupo (P2)	$(\tau_f + c)2^N - c \in \Theta(2^N \tau_f)$	Sí
Selección voraz (P2)	$kN - \frac{k(k-1)}{2} = \Theta(Nk)$ evaluaciones; $\Theta(Nk\tau)$ tiempo	Sí
Solución base completa	$\Theta(Nk\tau) + O(k + B \log k)$, más leer la entrada en $O(N + \sum_j \lvert S_j \rvert)$	No

6 Solución base

6.1 Organización del repositorio

Carpeta o archivo	Contenido
`VERSION`	Versión única del proyecto; se estampa en cada salida
`src/ejecutar_casos.py`	Programa principal: ejecuta los casos de `casos/`
`src/red/`	Algoritmos, validación de entrada, verificación y reportes
`src/completar_categorias.py`	Opcional: escribe en un archivo las categorías sintéticas que el programa genera al leer
`casos/`	Caso de la profesora, sin modificar; `ejemplos/` guarda casos de práctica
`resultados/`	Una carpeta por caso y el resumen de la ejecución
`tests/`	Pruebas automáticas (solo biblioteca estándar)
`docs/`	Manual técnico y manual de usuario
`Informe/`	Este informe (LaTeX y PDF), figuras y código de los experimentos

6.2 Flujo de una ejecución

1. **Lectura y validación** (`io_casos.py`). Acepta el formato de la profesora (`parametros`, `coeficiente_utilidad` y, si las hay, `categorias`) y un formato extendido. Si los nodos no traen categorías, como en el archivo de la profesora, se generan en memoria con semilla

fija (36921, universo de 20, entre 3 y 7 por nodo); el archivo no se modifica y la salida lo declara en `origen_categorias`. Rechaza con un mensaje claro: $k < 1$ o $k > N$, $B < 0$, ids repetidos, coeficientes no numéricos, categorías que no son una lista de textos y campos faltantes. Un caso inválido no detiene a los demás.

2. **Medición** (`pipeline.py`): mediana de 5 ejecuciones de selección y reparto, sin escribir trazas.
3. **Óptimos de referencia**: árbol exacto si $\binom{N}{k} \leq 500\,000$ y programación dinámica $O(kB^2)$ para el reparto si $kB^2 \leq 3 \cdot 10^6$.
4. **Resolución con traza**. El algoritmo es determinista: decide igual que al medir.
5. **Verificación** (`verificacion.py`): 12 chequeos (tabla siguiente).
6. **Salidas** por caso, en la carpeta `resultados/<caso>/`: `salida.json` (resultado, tiempo y verificación), `traza.json` y `traza.md` (selección con las ganancias marginales de todos los candidatos, reparto unidad por unidad y restricciones), y la misma traza en CSV. Además, `resultados/resumen.md`.

6.3 Verificación de restricciones

#	Chequeo	Qué garantiza
1	$\lvert A\rvert = k$	Cupo respetado
2	Nodos de A válidos y distintos	Selección bien formada
3	$f(A)$ de la traza $= f(A)$ recalculada	Cuenta incremental correcta
4	Ganancias de la selección no crecientes	Submodularidad observada
5	$f(A) \geq (1 - 1/e)\, f^*$ (si se calculó f^*)	Garantía de aproximación
6	x_i entero ≥ 0	Reparto factible
7	$x_i = 0$ si $i \notin A$	Solo reciben los elegidos
8	$\sum_i x_i \leq B$ (y $x_i \leq c_i$)	Presupuesto respetado
9	Utilidad recalculada coincide	Suma reportada correcta
10	Ganancias del reparto no crecientes	Concavidad observada
11	Ninguna transferencia de una unidad mejora	Optimalidad del reparto
12	No sobra presupuesto aprovechable	No se desperdician unidades útiles

El chequeo 11 basta para la optimalidad: con utilidades cóncavas, si mover una unidad de un nodo a otro nunca mejora, el reparto es un óptimo global (es la condición del argumento de intercambio de la Sección 4).

6.4 Correspondencia con la guía de la primera entrega

Requisito de la guía	Dónde está
Formalización de las dos partes (entradas, salidas, tamaño, objetivo y restricciones)	Sección 2
Prueba de NP-dureza por reducción desde Cobertura Máxima	Sección 3; `desde_cobertura_maxima` en `src/red/seleccion.py` y su prueba automática
Código de la solución base, dependencias, README e instrucciones	`src/`, `requirements.txt`, `pyproject.toml`, `README.md`
Manual técnico (algoritmo, estructura, formatos, pasos de ejecución, limitaciones)	`docs/manual_tecnico.md`
Ejecutar la solución base con los casos de la profesora	`casos/entrada_G2_red_15_nodos.json`, sin modificar; `python src/ejecutar_casos.py`
Resultados y tiempo por caso	`resultados/resumen.md` y `resultados/<caso>/salida.json`
Traza: selección, ganancias marginales, reparto y restricciones	`resultados/<caso>/traza.md` (partes A, B y C), `traza.json` y CSV
Reproducibilidad sin editar la lógica	El caso se agrega copiando el `.json` a `casos/`; semillas fijas
Misma versión en informe, repositorio, manual y evidencias	Archivo `VERSION`, estampado en cada salida; prueba automática

7 Resultados

7.1 Caso de la profesora

Caso `casos/entrada_G2_red_15_nodos.json`, tal como se recibió: $N = 15$, $k = 5$, $B = 30$ y $u_i(x) = w_i \ln(1 + x)$, con w_i el coeficiente de cada nodo dado por la profesora. La entrada trae solo los coeficientes, así que las categorías de datos son **sintéticas**: el programa las genera al leer el caso con semilla 36921, universo de 20 categorías y entre 3 y 7 por nodo, sin modificar el archivo. La salida lo declara en `origen_categorias`, y `src/completar_categorias.py` escribe esas mismas categorías en un archivo si se quieren ver.

Resultado	Valor
Nodos elegidos (en orden)	N04, N15, N14, N13, N06
Ganancias marginales Δ de la selección	7, 6, 4, 2, 1
Categorías cubiertas $f(A)$	20 de 20
Óptimo exacto	20, con $\binom{15}{5} = 3003$ evaluaciones; razón 1.0000
Reparto de $B = 30$	N04 = 7, N15 = 5, N14 = 6, N13 = 5, N06 = 7
Utilidad total	99.5821, igual al óptimo por programación dinámica para esos cinco nodos
Tiempo (selección + reparto)	0.018 ms (mediana de 5; depende del equipo)
Verificación	12 de 12 chequeos

Traza de la selección (`resultados/entrada_G2_red_15_nodos/traza.md`):

Paso	Elegido	Δ	Categorías nuevas	$f(A)$
1	N04	7	K01, K02, K06, K09, K12, K13, K14	7
2	N15	6	K03, K08, K10, K16, K17, K19	13
3	N14	4	K04, K07, K11, K15	17
4	N13	2	K05, K20	19
5	N06	1	K18	20

Los empates se resuelven por el coeficiente: en el paso 1 empatan con 7 N04, N06, N13, N14 y N15, y gana N04 ($w = 12.0$); en el paso 2 empatan N13 y N15 con 6, y gana N15 ($w = 9.5$ frente a 8.3); en el paso 5 empatan N05, N06 y N11 con 1, y gana N06 ($w = 11.1$). Las ganancias nunca suben, como predice la submodularidad.

Efecto del desempate. 33 grupos de cinco nodos cubren las 20 categorías; tras el reparto, su utilidad va de 91.06 a 107.04. La versión 1.0.0, que desempataba por posición en el archivo, entregaba N04, N13, N14, N03 y N05 con utilidad 92.58; desempatando por coeficiente, la cobertura es la misma y la utilidad sube a 99.58. La mejor de las 33 (107.04) no se alcanza porque selección y reparto se resuelven en secuencia (Sección 2).

Traza del reparto. La primera unidad va a N04 ($12.0 \ln 2 = 8.3178$), la segunda a N06 ($11.1 \ln 2 = 7.6939$), luego N14 (7.0008), N15 (6.5849) y N13 (5.7531). La sexta vuelve a N04 ($12.0(\ln 3 - \ln 2) = 4.8656$). Las 30 ganancias marginales aparecen en orden no creciente en `traza.md` y `traza_reparto.csv`.

7.2 Casos sintéticos de ejemplo

Casos de `casos/ejemplos/`, generados con semilla fija (`src/generar_casos_ejemplo.py`):

Caso	N	k	B	f	Razón vs. óptimo	Utilidad	ms
ej00_tres_nodos	3	2	6	5	0.8333 (5/6)	14.0511	0.006
ej01_pequeno	8	3	20	11	1.0000	55.3778	0.009
ej02_pequeno_caps	10	4	25	18	0.9474 (18/19)	34.1613	0.012
ej03_mediano	14	5	40	39	1.0000	80.4221	0.024
ej04_grande	500	20	2000	400	no se calcula	1221.8066	1.750
ej05_grande_caps	2000	40	10000	800	no se calcula	955.9874	11.686

Todos cumplen los 12 chequeos. En ej04 y ej05 el voraz cubre todo el universo (400 y 800 categorías), así que ahí también es óptimo. En los casos con óptimo calculado, la menor razón es 0.8333, muy por encima de la garantía de 0.632. Con el desempate por la ganancia de la primera unidad, la cobertura es la misma que con la versión 1.0.0 en todos los ejemplos y la utilidad no baja en ninguno (sube, por ejemplo, de 1054.64 a 1221.81 en ej04).

8 Validación experimental

Método. `Informe/code/experimentos.py` genera instancias sintéticas (semilla 2026) de 6 tamaños crecientes por experimento, duplicando el tamaño en cada paso. Cada punto es la mediana de 7 mediciones con `time.perf_counter`, tras una ejecución de calentamiento que no se mide y con el recolector de basura apagado. Luego se ajusta $t \approx c \cdot T(n)$ por mínimos cuadrados: si la deducción es correcta, el ajuste es casi perfecto ($R^2 \approx 1$) y el cociente $t/T(n)$ se mantiene casi constante. Los datos están en `Informe/code/experimentos.csv`.

Experimento	$T(n)$ teórico	c (s)	R^2	Crecimiento
Reparto, $N = 2000$, B de $2 \cdot 10^4$ a $6.4 \cdot 10^5$	$N + B \ln N$	$7.5 \cdot 10^{-8}$	0.9992	$B \times 32 \to t \times 29.5$
Reparto, $B = 20N$, N de 2000 a 64 000	$N + 20N \ln N$	$1.6 \cdot 10^{-7}$	0.9859	$N \times 32 \to t \times 89$
Selección, $k = 20$, N de 1000 a 32 000	Nk	$1.3 \cdot 10^{-7}$	0.9997	$N \times 32 \to t \times 32.6$
Selección, $N = 8000$, k de 5 a 160	Nk	$1.7 \cdot 10^{-7}$	0.9988	$k \times 32 \to t \times 40.5$

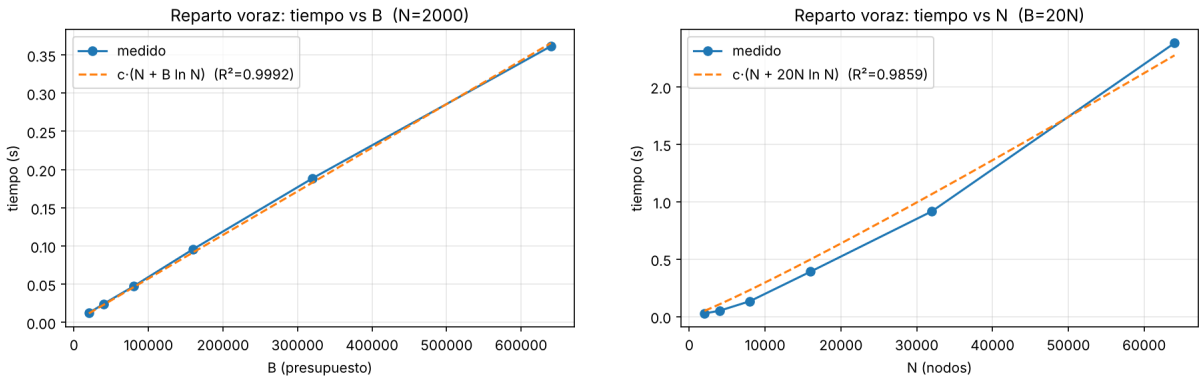


Figura 1: Reparto voraz: tiempo medido (puntos) frente a $c \cdot T(n)$ ajustado (línea).

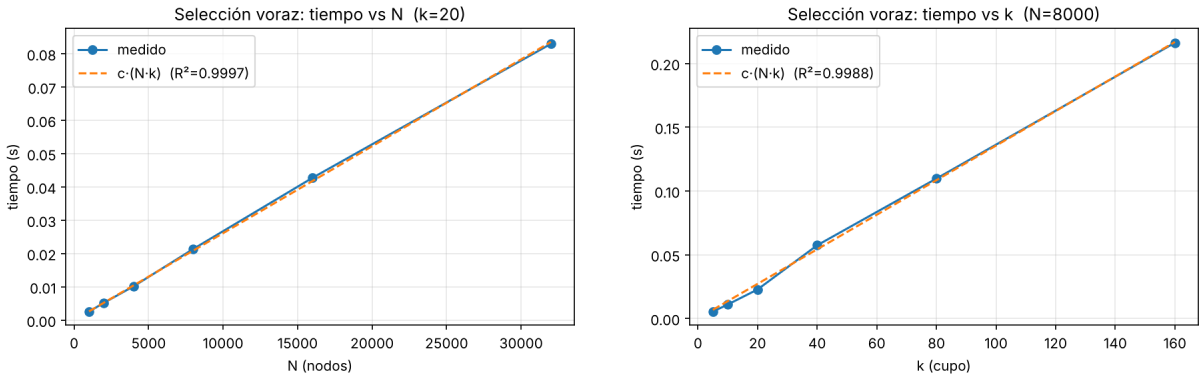


Figura 2: Selección voraz: tiempo medido frente a $c \cdot Nk$ ajustado.

Análisis.

- **Reparto contra B:** el tiempo es lineal en B ($\times 29.5$ para $B \times 32$; el cociente t/T pasa de $7.9 \cdot 10^{-8}$ a $7.4 \cdot 10^{-8}$), como predice el término $B \log N$.
- **Selección:** lineal en N y en k ($R^2 \geq 0.998$), con el desempate por peso incluido. El número de evaluaciones es exactamente $kN - k(k-1)/2$, comprobado por una prueba automática. Contra N el cociente t/T se mantiene entre 1.27 y $1.34 \cdot 10^{-7}$ ($\times 32.6$). Contra k crece algo más que $\times 32$ ($\times 40.5$; el cociente sube de 1.33 a $1.69 \cdot 10^{-7}$), por costos de memoria al recorrer listas más largas en cada paso; el orden de crecimiento no cambia.
- **Reparto contra N:** crece más que el modelo ($\times 89$ frente a $\times 46$ teórico). No es el algoritmo: `Informe/code/diagnostico_heap.py` mide con B fijo el costo por unidad, y de $N = 2000$ a 64 000 el reparto crece $\times 3.73$ y `heapq.heapreplace` aislado $\times 3.68$, frente a $\times 1.46$ esperado por $\log N$. El exceso está en el montículo de Python. Es coherente con que el análisis usa el modelo RAM (todo acceso cuesta lo mismo) y un montículo grande ya no cabe en la caché del procesador; esa causa física no se midió directamente. El crecimiento sigue muy lejos de uno cuadrático ($\times 1024$).

Exacto frente a voraz. Con $N = 40$ y $k = 20$, el exacto necesitaría $\binom{40}{20} \approx 1.4 \cdot 10^{11}$ evaluaciones (días de cómputo), mientras que el voraz resuelve $N = 32\,000$ con $k = 20$ en unos 0.08 s.

Referencias

[1] U. Feige. A threshold of $\ln n$ for approximating set cover. *Journal of the ACM*, 45(4):634–652, 1998.

[2] R. M. Karp. Reducibility among combinatorial problems. En *Complexity of Computer Computations*, pp. 85–103. Plenum Press, 1972.

[3] S. Khuller, A. Moss y J. Naor. The budgeted maximum coverage problem. *Information Processing Letters*, 70(1):39–45, 1999.

[4] G. L. Nemhauser, L. A. Wolsey y M. L. Fisher. An analysis of approximations for maximizing submodular set functions—I. *Mathematical Programming*, 14:265–294, 1978.

[5] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, 3.ª ed. MIT Press, 2009.

A Código de los algoritmos

El código completo está en `src/`. Para reproducir los resultados: `python src/ejecutar_casos.py`; para los experimentos y figuras de este informe: `python Informe/code/experimentos.py` (requiere `matplotlib` y `numpy`).

A.1 Parte 1: `src/red/reparto.py`

```python
"""PARTE 1 -- Reparto óptimo del presupuesto B entre N nodos con utilidades cóncavas.

Algoritmo voraz por ganancia marginal con montículo (cola de prioridad)::

    construcción del montículo ........ O(N)
    B extracciones/reinserciones ...... O(B log N)
    T(N, B) = O(N + B log N)           (= O(B log N) cuando B >= N)

La cota es pseudo-polinomial: B aparece como valor numérico, no como número de bits.
"""

from __future__ import annotations

import heapq
from typing import List, Optional, Sequence, Tuple

```

```python
from .utilidad import Utilidad

# (paso, nodo, ganancia_marginal, unidades_del_nodo_tras_asignar)
PasoReparto = Tuple[int, int, float, int]


def repartir_voraz(
    utilidades: Sequence[Utilidad],
    B: int,
    caps: Optional[Sequence[int]] = None,
    traza: Optional[List[PasoReparto]] = None,
) -> Tuple[List[int], float]:
    """Reparte hasta ``B`` unidades enteras maximizando la suma de utilidades.

    Args:
        utilidades: una utilidad cóncava y creciente por nodo.
        B: presupuesto total (entero >= 0).
        caps: tope opcional de unidades por nodo (presupuesto de ruido tolerable).
        traza: si se pasa una lista, se le agregan tuplas ``PasoReparto`` (una por unidad)
            .

    Returns:
        ``(x, total)``: ``x[i]`` unidades del nodo i y utilidad ganada sobre u_i(0).

    Complexity: O(N + B log N) tiempo, O(N) espacio.
    """
    if B < 0:
        raise ValueError("El presupuesto B debe ser >= 0.")
    N = len(utilidades)
    x = [0] * N
    total = 0.0
    # Montículo de MÍNIMOS de Python: se guarda -ganancia; el índice i rompe empates (
        menor primero).
    heap: List[Tuple[float, int]] = []
    for i, u in enumerate(utilidades):
        if caps is None or caps[i] > 0:
            heap.append((-u.marginal(0), i))
    heapq.heapify(heap)  # O(N)
    paso = 0
    while paso < B and heap:  # a lo sumo B iteraciones
        neg_g, i = heap[0]
        g = -neg_g
        if g <= 0:  # nadie aporta ya nada: se detiene
            break
        x[i] += 1
        total += g
        paso += 1
        if traza is not None:
            traza.append((paso, i, g, x[i]))
        if caps is not None and x[i] >= caps[i]:
            heapq.heappop(heap)  # nodo saturado: sale del montículo
        else:
            # Reemplaza el tope por la siguiente ganancia del mismo nodo en una sola
                operación O(log N).
            heapq.heapreplace(heap, (-utilidades[i].marginal(x[i]), i))
    return x, total


def repartir_dp(
    utilidades: Sequence[Utilidad],
```

```
    B: int,
    caps: Optional[Sequence[int]] = None,
) -> Tuple[List[int], float]:
    """Óptimo exacto por programación dinámica, O(N * B^2).

    Solo para VERIFICAR en instancias pequeñas que el voraz es óptimo.
    ``dp[i][b]`` = mejor utilidad usando los i primeros nodos con exactamente b unidades.
    """
    N = len(utilidades)
    NEG = float("-inf")
    dp = [[NEG] * (B + 1) for _ in range(N + 1)]
    elec = [[0] * (B + 1) for _ in range(N + 1)]
    dp[0][0] = 0.0
    for i in range(1, N + 1):
        u = utilidades[i - 1]
        tope = B if caps is None else min(B, caps[i - 1])
        for b in range(B + 1):
            for xi in range(0, min(b, tope) + 1):
                prev = dp[i - 1][b - xi]
                if prev == NEG:
                    continue
                v = prev + u.valor(xi)
                if v > dp[i][b]:
                    dp[i][b] = v
                    elec[i][b] = xi
    b_best = max(range(B + 1), key=lambda b: dp[N][b])
    x = [0] * N
    b = b_best
    for i in range(N, 0, -1):  # reconstrucción hacia atrás
        x[i - 1] = elec[i][b]
        b -= x[i - 1]
    return x, dp[N][b_best]
```

A.2 Parte 2: `src/red/seleccion.py`

```
"""PARTE 2 -- Selección de k nodos participantes (maximizar la utilidad conjunta f).

Implementación: f(A) = |unión de las categorías de los nodos de A| (cobertura: normalizada,
monótona y submodular). Cada conjunto de categorías se guarda como un entero-bitmask, un bit por
categoría del universo::

    ganancia marginal de j  =  popcount(S_j & ~cubierto)      -> costo tau = O(|U| / w)

* ``seleccion_voraz``: k pasos x N candidatos => kN - k(k-1)/2 evaluaciones = Theta(N k).
  Garantía (Nemhauser, Wolsey y Fisher, 1978): f(voraz) >= (1 - 1/e) * f(óptimo).
  Empates en la ganancia: se prefiere el nodo de mayor peso (ganancia de su primera unidad de
  presupuesto), para que a igual cobertura entren los nodos que más aprovechan el reparto.
* ``seleccion_exacta``: árbol "incluir / no incluir" con poda. Evalúa f exactamente C(N, k) veces
  (recurrencia de Pascal L(n, r) = L(n-1, r-1) + L(n-1, r)). Solo para N pequeño.
"""

from __future__ import annotations

import sys
from typing import Any, Dict, Iterable, List, Optional, Sequence, Tuple

```

```python
# Un paso de la traza de selección:
#   paso, nodo, ganancia (Delta), acumulado (f(A) tras agregarlo),
#   nuevas (bitmask de las categorías que aporta), candidatos ({nodo: Delta} de todos los evaluados)
PasoSeleccion = Dict[str, Any]


def f_cobertura(masks: Sequence[int], A: Iterable[int]) -> int:
    """Utilidad conjunta f(A): número de categorías distintas que cubren los nodos de A."""
    cub = 0
    for i in A:
        cub |= masks[i]
    return cub.bit_count()


def seleccion_voraz(
    masks: Sequence[int],
    k: int,
    traza: Optional[List[PasoSeleccion]] = None,
    pesos: Optional[Sequence[float]] = None,
) -> Tuple[List[int], int]:
    """Elige k nodos agregando en cada paso el de mayor ganancia marginal.

    Args:
        masks: bitmask de categorías de cada nodo.
        k: número de nodos a elegir, ``0 <= k <= N``.
        traza: lista opcional; recibe un ``PasoSeleccion`` por paso. Solo observa: no cambia
            ninguna decisión.
        pesos: criterio de desempate opcional, uno por nodo (en la solución base, la ganancia de la
            primera unidad de presupuesto, u_i(1) - u_i(0), que es proporcional al coeficiente w_i).

    Returns:
        ``(A, f(A))`` con A en orden de elección. Empates en la ganancia: gana el de mayor peso y, si
        también empatan (o no hay pesos), el índice menor. Si ya nadie aporta, se siguen agregando
        nodos hasta |A| = k. El desempate no cambia f(A) en cada paso, así que la garantía
        1 - 1/e se mantiene.

    Raises:
        ValueError: si k no está en [0, N] o ``pesos`` no tiene N valores.

    Complejidad: Theta(N k) evaluaciones de la ganancia, cada una O(|U| / w).
    """
    N = len(masks)
    if k < 0 or k > N:
        raise ValueError(f"k debe cumplir 0 <= k <= N (k={k}, N={N}).")
    if pesos is not None and len(pesos) != N:
        raise ValueError(f"se esperaban {N} pesos y llegaron {len(pesos)}.")
    peso = pesos if pesos is not None else [0.0] * N
    elegido = [False] * N
    cubierto = 0
    A: List[int] = []
    for paso in range(1, k + 1):
        mejor, mejor_g = -1, -1
        candidatos: Dict[int, int] = {}
```

```
        for j in range(N):  # O(N) candidatos por paso
            if elegido[j]:
                continue
            g = (masks[j] & ~cubierto).bit_count()  # Delta_j(A): categorías nuevas de j
            if traza is not None:
                candidatos[j] = g
            # '>' estricto: con igual ganancia y peso se queda el índice menor
            if g > mejor_g or (g == mejor_g and peso[j] > peso[mejor]):
                mejor, mejor_g = j, g
        nuevas = masks[mejor] & ~cubierto
        elegido[mejor] = True
        cubierto |= masks[mejor]
        A.append(mejor)
        if traza is not None:
            traza.append({"paso": paso, "nodo": mejor, "ganancia": mejor_g,
                          "acumulado": cubierto.bit_count(), "nuevas": nuevas, "candidatos
                              ": candidatos})
    return A, cubierto.bit_count()


def seleccion_exacta(masks: Sequence[int], k: int) -> Tuple[List[int], int, int]:
    """Óptimo exacto con el árbol "incluir / no incluir" el nodo i, con poda.

    Casos de ``explorar(i, A)``:
      1. |A| = k: hoja; se evalúa f(A) desde cero (tau = O(k |U| / w)).
      2. quedan menos nodos que lugares libres: rama sin solución (poda).
      3. se ramifica: (a) incluir i, (b) no incluir i; gana la mejor (empate: la de
          incluir).

    Returns:
        ``(A*, f(A*), evaluaciones)``; ``evaluaciones`` es siempre C(N, k).

    Raises:
        ValueError: si k no está en [0, N].

    Complejidad: C(N, k) evaluaciones de f; O(C(N, k) (N + tau)) tiempo. Solo para N peque
        ño.
    """
    N = len(masks)
    if k < 0 or k > N:
        raise ValueError(f"k debe cumplir 0 <= k <= N (k={k}, N={N}).")
    sys.setrecursionlimit(max(sys.getrecursionlimit(), 2 * N + 100))  # profundidad máxima
        : N
    evaluaciones = 0
    A: List[int] = []

    def explorar(i: int) -> Tuple[int, List[int]]:
        nonlocal evaluaciones
        if len(A) == k:  # caso 1: cupo lleno
            evaluaciones += 1
            return f_cobertura(masks, A), list(A)
        if N - i < k - len(A):  # caso 2: poda
            return -1, []
        A.append(i)  # caso 3a: incluir i
        v1, A1 = explorar(i + 1)
        A.pop()
        v2, A2 = explorar(i + 1)  # caso 3b: no incluir i
        return (v1, A1) if v1 >= v2 else (v2, A2)

    valor, mejor = explorar(0)
```

```python
    return mejor, valor, evaluaciones


def desde_cobertura_maxima(
    universo: Iterable[Any],
    subconjuntos: Sequence[Iterable[Any]],
    k: int,
    t: int,
) -> Tuple[List[int], int, int]:
    """Reducción Cobertura Máxima -> Selección de participantes (prueba de NP-dureza).

    Un nodo por subconjunto, f(A) = |unión|, cupo k y umbral t; se construye en O(sum |S_i|).
    Devuelve ``(masks, k, t)``, la instancia de selección equivalente.
    """
    idx = {e: j for j, e in enumerate(universo)}
    masks = [sum(1 << idx[e] for e in set(S)) for S in subconjuntos]
    return masks, k, t
```